Data Preprocessing

Scaling, encoding and honest splits

The problem

- Features live on wildly different scales
- Models need numbers, not category labels
- We must measure performance on unseen data

Standard scaling

$$z = \frac{x - \mu}{\sigma}$$

- Fit μ and σ on the training set only
- A constant column gets σ = 1 to avoid ÷0
- Helps gradient descent, k-NN, SVM, PCA

StandardScaler

```
class StandardScaler:
    def fit(self, X):
        X = np.asarray(X, float)
        self.mean_, self.std_ = X.mean(0), X.std(0)
        self.std_[self.std_ == 0] = 1
        return self

    def transform(self, X):
        return (np.asarray(X, float) - self.mean_) / self.std_
```

One-hot encoding

- Class 2 of 4 → [0, 0, 1, 0]
- No fake ordering between categories
- Also the target format for softmax

one_hot

```
def one_hot(y, n_classes=None):
    y = np.asarray(y, int)
    n = n_classes or y.max() + 1
    out = np.zeros((len(y), n))
    out[np.arange(len(y)), y] = 1
    return out
```

Honest evaluation

- Shuffle, then cut: train 75% / test 25%
- k-fold: every point validates exactly once
- Fixed seed → reproducible experiments

Split and k-fold

```python
def train_test_split(X, y, test_size=0.25, seed=0):
    idx = np.random.default_rng(seed).permutation(len(X))
    cut = int(len(X) * (1 - test_size))
    return X[idx[:cut]], X[idx[cut:]], y[idx[:cut]], y[idx[cut:]]

def k_fold_indices(n, k, seed=0):
    idx = np.random.default_rng(seed).permutation(n)
    folds = np.array_split(idx, k)
    return [(np.concatenate(folds[:i] + folds[i + 1:]), folds[i]) for i in range(k)]
```

Recap

- Scale with training statistics only
- One-hot for categories
- Shuffle + split, or k-fold, for honest scores
- All operations are $O(n \cdot d)$